



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر
گزارش پروژه درس سیستم عامل

عنوان:

پیاده‌سازی سیستم build در داکر

نگارش

سارا قضاوی

آرش قوامی

سید معین حسینی

زهرا قصابی

استاد

دکتر حسین اسدی

بهمن ماه سال ۱۴۰۴



چکیده: هدف این پروژه آشنایی عملی با نحوه ساخته شدن ایمیج‌ها در داکر و درک بهتر سازوکار داخلی دستور build است. در این پروژه ابزار zocker را تغییر داده‌ایم، به صورتی که فرآیند build ایمیج‌ها را شبیه‌سازی می‌کند و مکانیزم caching را نیز اعمال می‌کند.

۱ بخش اول

در بخش اول پروژه به لایه‌های ایمیج می‌پردازیم. می‌دانیم که در داکر هر ایمیج چند لایه دارد و هر دستور از Dockerfile در اغلب موارد یک لایه جدید می‌سازد. ما در زاکر مانند همین مکانیزم بر اساس Zockerfile یک ایمیج را می‌سازیم. این فایل شامل دستوراتی است که نحوه build شدن ایمیج را مشخص می‌کند. دستورات پیاده‌سازی شده در این پروژه عبارتند از:

FROM, BASEDIRE, RUN, COPY, ADD, WORKDIR, ARG, CMD

بخش های پیاده‌سازی شده به شرح زیر است:

در ابتدا در فایل config.h به استراکت config فیلدهایی برای نام ایمیج، مسیر Zockerfile و آرگومان‌های build ایجاد می‌کنیم:

```
struct config {
    enum COMMAND subcommand;
    char name[64];
    char command[256];
    char base_dir[512];
    char base_image[1024];

    char image_name[128];      // -t in build
    char zockerfile_path[512]; // -f in build

    // build args passed from CLI: --build-arg KEY=VAL
    struct build_kv build_args[MAX_BUILD_ARGS];
    size_t build_argc;
};
```

توجه شود که استراکت build_kv برای ذخیره نام و مقدار متغیرهایی است که در دستور build ذکر شده‌اند.

در فایل config.c وجود آرگومان‌های دستور را بررسی می‌کنیم:

```
if (cfg.subcommand == BUILD) {
    if (strcmp(cfg.image_name, "") == 0) {
        fprintf(stderr, "[ERR] Missing image name (-t <name>)\n");
        return 1;
    }
    if (strcmp(cfg.zockerfile_path, "") == 0) {
        fprintf(stderr, "[ERR] Missing zockerfile path (-f <path>)\n");
        return 1;
    }
}
```

از آنجا که باید مسیر working dir را نگهداری کنیم و همچنین برای دستور CMD، مسیر فعلی را بدانیم، در فایل run.h این دو فیلد را اضافه می‌کنیم:

```
struct container {
    char id[64];
    char command[256];
    char base_dir[512];
    char base_image[1024];

    char workdir[256];
    char default_cmd[256];
};
```

در فایل run.c حالتی که base-dir تعریف نشود و از ایمج پایه استفاده شود را اضافه می‌کنیم:

```

} else {
    // resolve and mount filesystem from base image
    if (setup_container_dir(cont.id, container_dir, cont.base_image,
        cont.workdir, sizeof(cont.workdir), cont.default_cmd,
        sizeof(cont.default_cmd)) != 0) {
        fprintf(stderr, "[ERR] Failed to setup container directory for %s\n"
            cont.id);

        return 1;
    }
}
}

```

همچنین مسیر اجرای دستور را بر اساس اولویت (دستور کاربر، CMD ایمیج، و یا دیفالت سیستم) تعیین می‌کنیم:

```

const char *final_cmd = NULL;
if (cont.command[0] != '\0') final_cmd = cont.command;
else if (cont.default_cmd[0] != '\0') final_cmd = cont.default_cmd;
else final_cmd = "/bin/sh";

```

در ادامه فایل setup.c را تغییر می‌دهیم. این فایل مسئول آماده‌سازی فایل سیستم کانتینر و اتصال آن به لایه‌های ایمیج پایه است. در ابتدا با توجه به نوع ایمیج، در تابع resolve_image_spec سه حالت ایمیج داخلی زاگر (با پیشوند zimg)، مسیر دایرکتوی absolute، و ایمیج docker (با دستور docker image inspect و ابزار jq) هندل می‌شوند:

```

// zimg:<name>
if (is_zimg(spec)) {
    ...
    if (read_kv(manifest, "final_lowerdirs", lowerdirs, lower_sz) != 0)
    {...}

    (void)read_kv(manifest, "final_workdir", workdir, workdir_sz);
    (void)read_kv(manifest, "final_cmd", cmd, cmd_sz);
}

```

```

    return 0;
}

// absolute path: treat as already-lowerdir string
if (spec[0] == '/') {
    strncpy(lowerdirs, spec, lower_sz - 1);
    lowerdirs[lower_sz - 1] = 0;
    return 0;
}

// docker image (use image's overlay2 LowerDir chain)
if (get_docker_image_lowerdirs(spec, lowerdirs, lower_sz) != 0)
    {...}

```

پس از تعیین lowerdir دایرکتوری ایمپج ساخته می‌شود و مطابق ساختار زاگر، پوشه‌های لازم ایجاد می‌شوند.

در ادامه فایل zockerfile.h را ایجاد می‌کنیم:

```

enum zf_op {
    ZF_FROM,
    ZF_BASEDIR,
    ZF_RUN,
    ZF_COPY,
    ZF_ADD,
    ZF_WORKDIR,
    ZF_ARG,
    ZF_CMD,
    ZF_UNKNOWN
};

struct zf_instr {
    enum zf_op op;
    char raw[1024]; // full trimmed line
    char arg[1024]; // everything after keyword

```

```

    int line_no;
};

struct zf_program {
    struct zf_instr *items;

    size_t len;

    size_t cap;
};

```

استراکت `zf_instr` برای نگهداری هر دستور در `Zockerfile` و مقادیر آن است. استراکت `zf_program` تمامی دستورات را به عنوان یک برنامه زاکر نگهداری می‌کند.

در فایل `zockerfile.c`، تابع کلیدی `zf_parse_file` تعریف شده است. این تابع یک `Zockerfile` را می‌خواند و دستورات موجود در آن را بر اساس `enum` تشخیص می‌دهد و سپس استراکت هر دستور را می‌سازد و به استراکت کلی برنامه اضافه می‌کند. این تابع به صورت زیر پیاده‌سازی شده است:

```

int zf_parse_file(const char *path, struct zf_program *out) {
    FILE *fp = fopen(path, "r");

    ...

    // create new instructor and add to program
    struct zf_instr *it = &out->items[out->len++];

    memset(it, 0, sizeof(*it));

    it->op = parse_op(kw);

    it->line_no = line_no;

    memcpy(it->raw, line, strlen(line) + 1);

    memcpy(it->arg, rest, strlen(rest) + 1);

    ...
}

```

سپس فایل‌های `build.h` و `build.c` را ایجاد می‌کنیم. فایل `build.c` وظیفه اصلی در `build` کردن ایمیج‌ها را بر عهده دارد. ساختار کلی دستور `build` به صورت زیر پیاده شده است:

برای هر ایمیج، به ازای دستوراتی که نیاز به ساخت لایه جدید دارند، یک پوشه با `hash` مخصوص

هر لایه در مسیر global برای تمام ایمیج‌ها ساخته می‌شود. سپس با نام ایمیج، یک دایرکتوری ایجاد می‌شود که در فایل manifest.txt اطلاعات ایمیج شامل نام آن، مسیر Zockerfile، دستورات آن و hash هر لایه‌ای که به ازای هر دستور ساخته شده ذخیره می‌شوند. در انتهای این فایل نیز final_lowerdir ذخیره می‌شود تا بتوان ترتیب لایه‌های ایمیج را تشخیص داد. با این روش می‌توان لایه‌ها را بین ایمیج‌های مختلف به اشتراک گذاشت و از ایجاد کپی‌های زیاد جلوگیری کرد که برای مکانیزم caching نیز کاربردی است.

حال به بررسی بدنه اصلی می‌پردازیم. هر یک از دستورات مربوط به Zockerfile به صورت جداگانه در تابع build_image مدیریت می‌شوند. در ابتدای تابع، mount ها را ایزوله‌سازی می‌کنیم. این کار باعث می‌شود وارد یک mount namespace جدید شویم و تغییرات mount فقط در این build دیده شود. پس از ساختن دایرکتوری ایمیج و فایل اطلاعات، با تابع parser گفته شده فایل را به استراکتی از دستورات ترجمه می‌کنیم و آن را validate می‌کنیم. به عنوان مثال، هر فایل فقط یک دستور FROM/BASEDIR دارد که در ابتدای فایل است. سپس بر اساس دستور اول، ایمیج ساخته می‌شود (از دایرکتوری یا فایل ایمیج) در ادامه در یک حلقه دستورات را یکی یکی بررسی و اجرا می‌کنیم.

• دستورات FROM و BASEDIR (خارج حلقه):

ابتدا بررسی می‌کنیم که در ابتدای تابع کلمه FROM آمده است یا BASEDIR. اگر کلمه BASEDIR آمده بود، از مسیر پوشه مستقیماً به عنوان lowerdir استفاده می‌شود:

```
if (it->op == ZF_BASEDIR) {
    strncpy(current_lowerdirs, base_spec,
            sizeof(current_lowerdirs) - 1);
    current_lowerdirs[sizeof(current_lowerdirs) - 1] = 0;
}
```

در غیر این صورت، اگر کلمه FROM در ابتدای فایل آمده بود، با استفاده از تابع resolve_image_spec لایه‌های ایمیج را پیدا می‌کنیم. مسیر lowerdir ها نیز داخل متغیر current_lowerdirs قرار می‌گیرد:

```
if (resolve_image_spec(base_spec, current_lowerdirs,
```



```
sizeof(current_lowerdirs), NULL, 0, NULL, 0) != 0)
{...}
```

اطلاعات base نیز داخل manifest.txt ذخیره می‌شود.

- دستور WORKDIR:

با این دستور پوشه‌ای که دستورات در آن اجرا می‌شوند تغییر می‌یابد. ابتدا بررسی می‌کنیم که آدرس absolute باشد و سپس پوشه را تغییر می‌دهیم.

- دستور RUN:

این دستور، یک عملیات shell را داخل لایه جدید اجرا می‌کند.

- دستور CMD:

این دستور لایه جدیدی نمی‌سازد. صرفاً metadata را تغییر می‌دهد که این تغییر با عوض کردن مقدار متغیر final_cmd انجام می‌شود. در انتها نیز مقدار final_cmd داخل manifest نوشته می‌شود.

- دستور COPY:

در این دستور ابتدا بررسی می‌کنیم که آیا relative path داده شده صحیح است یا خیر (توجه داریم که در این دستور ورودی دادن آدرس absolute مجاز نیست). سپس با استفاده از آن absolute_path را می‌سازیم. در ادامه نیز با استفاده از تابع access به فایل دسترسی پیدا می‌کنیم و آن را در مسیر مشخص شده کپی می‌کنیم و فایل manifest را به‌روزرسانی می‌کنیم.

- دستور ADD:

این دستور دو حالت دارد: در حالت اول، اگر آدرس روی سیستم باشد، عملکردی مشابه COPY خواهد داشت. در حالت دوم، اگر آدرس وارد شده http باشد، با استفاده از curl فایل را از URL دانلود کرده و داخل مسیر نهایی لایه ذخیره می‌کنیم.

- دستور ARG:

با اجرای این دستور، یک زوج key, value در آرایه args ذخیره می‌شود. حال اگر در دستورات بعدی از کلمه‌ای که در آرایه ذخیره شده بود استفاده شود (مثلاً \$KEY)، با مقدار VALUE جایگزین می‌شود. توجه داریم که استفاده از ARG لایه جدیدی نمی‌سازد و صرفاً یک key, value در حافظه ذخیره می‌کند.

پیاده‌سازی دستوراتی که نیاز به ساختن یک لایه جدید دارند، با استفاده از یک تابع مشترک انجام

شده است:

```
static int apply_layer(..., const char *instruction,
                      layer_action_fn action, ...) {
    ...
    snprintf(opts, sizeof(opts), "lowerdir=%s,upperdir=%s,workdir=%s",
    ↪      current_lowerdirs, lp.diff_dir, lp.work_dir);
    // mount
    int rc = action(lp.merged_dir, ctx);
    // unmount
    if (store_layer_globally(&lp) != 0) {...}
    if (add_cached_layer_to_chain(current_lowerdirs, lp.diff_dir) != 0)
        {...}
    ...
}
```

در این تابع، برای هر دستور اطلاعات آن در opts نوشته می‌شود و سپس با استفاده از تابع action دستور اجرا می‌شود و لایه آن در پوشه global ساخته شده و زنجیره cache اضافه می‌شود. توجه شود که تابع action برای هر دستور جداگانه با استفاده از یک استراکت تعریف می‌شود. به عنوان مثال:

```
struct act_run {
    const char *cmd;
    const char *workdir;
};

static int action_run(const char *merged_dir, void *ctx) {
    struct act_run *a = (struct act_run *)ctx;
    return run_in_chroot(merged_dir, a->workdir, a->cmd);
}
```

۲ بخش دوم: پیاده‌سازی مکانیزم Cache

در این بخش، مکانیزم Layer Caching مستقیماً به تابع `build_image` اضافه شده است. هدف این مکانیزم جلوگیری از اجرای مجدد مراحل تکراری در فرآیند `build` و بازاستفاده از لایه‌های از قبل ساخته‌شده است.

فایل‌های `hash.h` و `hash.c` را ایجاد می‌کنیم. در این فایل تابع مهم `layer_hash` پیاده‌سازی شده که مقدار `hash` را به صورت یکتا برای هر لایه ایجاد می‌کند. در `apply_layer`، ابتدا برای هر لایه مقدار هش آن قبل از ساخت Overlay محاسبه می‌شود تا مشخص شود آیا لایه مشابه قبلاً ساخته شده است یا خیر. اگر قبلاً ساخته شده باشد، از پوشه آن لایه در پوشه اصلی (`global`) استفاده می‌شود. یعنی:

- Overlay جدید ساخته نمی‌شود.
- دستور اجرا نمی‌شود.
- مسیر `diff` همان لایه مستقیماً وارد زنجیره `lowerdirs` می‌شود.

در این حالت، لایه به عنوان `Cache Hit` در نظر گرفته می‌شود و زمان بیلد به شدت کاهش می‌یابد.

در صورت عدم وجود هش دایرکتوری‌های `diff` و `work` ساخته می‌شوند و `mount Overlay` می‌شود. دستور در محیط `chroot` اجرا می‌شود و لایه جدید ثبت و به ساختار کش اضافه می‌شود. به دلیل استفاده از `Hash` زنجیره‌ای، کش به صورت مرحله‌ای عمل می‌کند. اگر در `Zockerfile` تنها یکی از دستورات میانی تغییر کند، تمام مراحل قبل از آن از کش استفاده می‌کنند و فقط همان مرحله و مراحل بعدی مجدداً ساخته می‌شوند.

در ادامه دستورات `history`، `images` و `rmi` پیاده‌سازی می‌شوند. دستور `history` بر اساس داده‌های موجود در `manifest.txt` اجرا می‌شود. دستور `images` نیز در پوشه ایمیج‌ها تمام دایرکتوری‌ها را باز می‌کند و بر اساس فایل `manifest.txt` آنها لیستی از ایمیج‌ها می‌سازد. دستور `rmi`، فقط دایرکتوری ایمیج که یک فایل اطلاعات دارد را پاک می‌کند. از آنجا که لایه‌ها در یک پوشه `global` هستند، این عمل بر سایر ایمیج‌ها تاثیری نمی‌گذارد.

دستور نهایی این بخش دستور `prune` است. این دستور برای حذف لایه‌های بلا استفاده به کار می‌رود. در این دستور:

۱. تمام ایمیج‌های موجود بررسی می‌شوند.

۲. لایه‌های در حال استفاده مشخص می‌شوند (با استفاده از hash های موجود در مسیر lowerdir).

۳. لایه‌هایی که توسط هیچ ایمیجی ارجاع نشده‌اند حذف می‌شوند.

به دلیل وجود وابستگی زنجیره‌ای، حذف به صورت تکراری انجام می‌شود تا زمانی که هیچ لایه قابل حذف دیگری باقی نماند.

تمام دستورات این مرحله در main.c اضافه شده‌اند. همچنین فایل‌های history.h و history.c برای اجرای دستور history و parse کردن فایل manifest.txt به استراکتی از تاریخچه ایمیج ایجاد شده‌اند. برای اجرای دستور prune، فایل image_cmd.c ایجاد شده است که فرآیند حذف را به صورت تکرار شده انجام می‌دهد.

۳ امتیازی

در این بخش از پروژه، قابلیت multi-stage build به سیستم build ابزار Zocker افزوده شده‌است. هدف از پیاده‌سازی این بخش، افزایش انعطاف‌پذیری فرآیند ساخت ایمیج‌ها، کاهش حجم ایمیج نهایی و جداسازی محیط ساخت از محیط اجراست. این قابلیت مشابه مکانیزم multi-stage build در Docker طراحی شده و به کاربر اجازه می‌دهد چند مرحله مجزا از ساخت ایمیج را در یک فایل Zockerfile تعریف کند.

در این بخش مکان تعریف چند مرحله مستقل در فرآیند ساخت ایمیج فراهم شده است، به گونه‌ای که هر مرحله می‌تواند از یک ایمیج پایه متفاوت شروع شود و خروجی مراحل قبلی در مراحل بعدی مورد استفاده قرار گیرد. هدف اصلی این بخش این است که ابزارهای موردنیاز در زمان ساخت در ایمیج نهایی باقی نمانند و تنها فایل‌های ضروری برای اجرا در خروجی نهایی قرار گیرند. همچنین این قابلیت باید با معماری لایه‌ای ایمیج‌ها و مکانیزم کش پیاده‌سازی شده در بخش‌های قبلی پروژه سازگار باشد.

برای پیاده‌سازی این قابلیت، ابتدا پردازش فایل Zockerfile توسعه داده شد تا امکان شناسایی چند دستور FROM/BASEDIR در یک فایل فراهم شود. هر دستور FROM/BASEDIR به عنوان شروع یک مرحله جدید در نظر گرفته می‌شود و در صورت وجود عبارت AS، نام مشخصی به آن مرحله اختصاص داده می‌شود که در مراحل بعدی قابل استفاده است. دستورات بین دو FROM/BASEDIR

متوالی به همان مرحله تعلق دارند و به صورت مستقل از سایر مراحل پردازش می شوند. به این ترتیب، فایل Zockerfile به مجموعه ای از مراحل ساخت مجزا تقسیم می شود که هر یک دارای زمینه اجرای مستقل است.

پس از شناسایی مراحل مختلف، برای هر مرحله یک محیط ساخت مستقل ایجاد می شود. هر مرحله دارای زنجیره مخصوص به خود از لایه ها، فایل سیستم مبتنی بر OverlayFS، مسیر کاری و متغیرهای مربوط به build است. این ایزوله سازی باعث می شود تغییرات ایجاد شده در یک مرحله به صورت مستقل به مراحل دیگر منتقل نشود و تنها در صورت درخواست صریح کاربر، امکان استفاده از خروجی یک مرحله در مرحله دیگر وجود داشته باشد. این ساختار، رفتار سیستم را به معماری multi-stage در سیستم های کانتینری واقعی نزدیک تر می کند.

یکی از بخش های کلیدی multi-stage build، امکان انتقال فایل ها بین مراحل مختلف است. برای این منظور، دستورات ابا گزینه from- توسعه داده شده اند. هنگام برخورد با این فلگ، سیستم build ابتدا مرحله مرجع را بر اساس نام مشخص شده پیدا می کند و سپس فایل سیستم نهایی آن مرحله را به عنوان منبع در نظر می گیرد. فایل یا مسیر مورد نظر از خروجی مرحله قبلی استخراج شده و به فایل سیستم مرحله فعلی منتقل می شود. این عملیات نیز منجر به ایجاد یک لایه جدید می گردد و به صورت کامل با مکانیزم لایه بندی سیستم یکپارچه است.

پس از اجرای تمام مراحل تعریف شده در فایل Zockerfile، تنها آخرین مرحله به عنوان ایمیج نهایی در نظر گرفته می شود و نام و تگ مشخص شده توسط کاربر به آن اختصاص داده می شود. مراحل میانی به عنوان ایمیج های موقتی تلقی می شوند و در خروجی دستور zocker images نمایش داده نمی شوند.